



Verification and Validation Report: RoCam

Team #3, SpaceY

Zifan Si

Jianqing Liu

Mike Chen

Xiaotian Lou

1 Revision History

Date	Version	Notes
2026-03-04	1.0	Add Document Evaluation
2026-03-05	1.1	Add Unit Tests based on run

2 Symbols, Abbreviations and Acronyms

symbol	description
T	Test

Contents

1	Revision History	i
2	Symbols, Abbreviations and Acronyms	ii
3	Document Evaluation	1
3.1	SRS Evaluation	1
3.1.1	Abstraction	1
3.1.2	Testability	1
3.1.3	Ambiguity	2
3.1.4	Completeness	2
3.1.5	Spelling and Grammar	3
3.1.6	Ease of Understanding	3
3.2	Design Evaluation	3
3.2.1	Consistency With Requirements	3
3.2.2	Modularity and Interfaces	3
3.2.3	Maintainability	3
3.3	Verification and Validation Plan Evaluation	4
3.3.1	Completeness	4
3.3.2	Unambiguous	4
3.3.3	Feasibility	4
3.3.4	Overall Objective Accomplishment	4
4	Functional Requirements Evaluation	4
4.1	Scope and Execution Policy	5
4.2	Summary of Planned Functional Tests vs Execution	5
4.3	Unit-Test Evidence for Functional Requirements	6
4.3.1	Evidence Format and Reproducibility	6
4.4	Stakeholder Validation Touchpoint	6
5	Nonfunctional Requirements Evaluation	6
5.1	Usability	6
5.2	Performance	6
5.3	etc.	6
6	Comparison to Existing Implementation	6
7	Unit Testing	7
7.1	Backend Unit Tests (pytest)	7
7.2	Frontend Unit Tests (Vitest)	15
8	Changes Due to Testing	20
9	Automated Testing	20

10 Trace to Requirements	20
11 Trace to Modules	20
12 Code Coverage Metrics	20

List of Tables

1 Functional Test Execution Summary (traceable to VnV Plan)	5
---	---

List of Figures

This document ...

3 Document Evaluation

3.1 SRS Evaluation

3.1.1 Abstraction

The [SRS](#) was reviewed to ensure requirements describe *what* RoCam must do from a stakeholder/operator perspective (camera operator, rocketry team) rather than *how* it is implemented (specific CV models, firmware details, or UI framework choices). Overall, the Functional Requirements (FRs) are written at a suitable level of abstraction: they describe externally observable behaviours such as acquiring frames, displaying a live preview, allowing manual control, tracking/arming transitions, and recording management.

The Non-Functional Requirements (NFRs) were also reviewed and are generally abstract (e.g., usability, performance, safety, reliability). Where an NFR implicitly assumes a particular technical solution (e.g., a specific fps target or a specific latency measurement method), this is acceptable if the [SRS](#) also defines a clear, measurable fit criterion.

3.1.2 Testability

The FRs are mostly testable because they describe observable outcomes that can be checked via logs, UI state, or recorded artifacts (e.g., a created video file). However, due to the final project scope constraint that **only unit testing is executed** (and system/field tests are out of scope), some FRs are not directly testable end-to-end within this iteration. To preserve testability under the reduced scope, FR evaluation in this project is performed through:

- Unit tests for isolated logic (e.g., state transitions, validation, file naming/path logic, error formatting),
- Component-level tests where a module boundary exists (e.g., UI state reducers, API request formatting, parsing),
- Stakeholder review feedback for user-facing correctness/priority (without live flight validation).

For the NFRs, testability varies:

- **Performance:** testable if the [SRS](#) states measurable thresholds (e.g., 1080p60 I/O, latency targets), but in this project we can only partially test performance via unit/component tests and profiling of specific pipelines rather than full system measurement.
- **Usability:** testable through walkthrough protocols and checklists (screenshots, task completion observations), but results are less precise than instrumentation-based studies.

- **Safety:** testable for software guards (confirmation prompts, manual arming only, safe idle transitions) via unit tests and static inspection; physical safety behaviours at the gimbal/hardware layer are limited by out-of-scope hardware validation.

3.1.3 Ambiguity

The [SRS](#) wording is largely clear, but several items can be improved to reduce interpretation differences during verification:

- **Terms that need operational definitions:** phrases such as “real-time”, “smooth”, “fluid”, “clear error message”, and “maintains lock” should map to concrete thresholds (fps, latency, allowed dropped frames, maximum time-to-reacquire, required UI elements).
- **State-related FRs:** FRs related to *armed*, *tracking*, and *idle* benefit from explicitly defining entry/exit conditions and time bounds (e.g., “within 2s of sustained loss, transition to idle”).
- **Recording management:** where FRs mention “download” or “list”, the expected metadata should be explicit (name format, timestamp source, duration computation, sorting rule).

These ambiguities do not block implementation but do increase the risk of disagreement during V&V. Where possible, they are handled in this project by encoding assumptions as unit-testable acceptance criteria.

3.1.4 Completeness

For completeness, we reviewed whether the [SRS](#) covers the primary stakeholder goals for RoCam: acquisition, operator control, automated tracking behaviour, recording workflow, and safe operation. At a high level, the [SRS](#) is close to complete for Rev0 scope. The main completeness concerns arise from:

- **Out-of-scope verification constraints:** since system tests and field tests are not executed in this iteration, any FR/NFR that requires integrated hardware measurement (true end-to-end latency, outdoor robustness, real launch tracking) cannot be fully verified. The [SRS](#) should explicitly mark such requirements as requiring future work or later verification stages.
- **Third-party dependencies:** camera/gimbal behaviour is assumed per manufacturer documentation; the [SRS](#) should clarify which behaviours are *assumed* vs *verified by this team*.
- **Safety and hazard linkage:** any hazard mitigations (e.g., emergency stop, confirmation prompts, manual arm gating) should be explicitly referenced in either FRs or NFRs so that traceability from the Hazard Analysis to requirements is direct.

3.1.5 Spelling and Grammar

A pass over the [SRS](#) found no major grammar defects; remaining issues are primarily minor consistency fixes (capitalization, hyphenation, consistent use of abbreviations such as “UI”, “fps”, “gimbal”). In the final report, we standardize terminology across documents ([SRS/MG/MIS/VnV Plan/VnV Report](#)) to reduce confusion during traceability review.

3.1.6 Ease of Understanding

The [SRS](#) is readable and structured, but ease-of-understanding can be improved by:

- Making requirement IDs clickable links in the PDF (or ensuring the traceability tables include enough context so readers do not need to search manually).
- Adding brief legends/notes to diagrams (if any) explaining symbols and data flow, especially when multiple modules interact (Jetson pipeline, STM32 controller, UI).
- Including a short glossary of user-facing terms (armed/tracking/idle, “preview”, “overlay”, “recording”) so interpretation is consistent.

3.2 Design Evaluation

3.2.1 Consistency With Requirements

We reviewed the [MG/MIS](#) against the [SRS](#) to ensure that each major requirement has a clear design owner (vision pipeline, control firmware/interface, UI/recording). Overall, the decomposition is aligned with RoCam’s core functions. The main design-to-requirement risk is where requirements imply integrated behaviour (e.g., tracking quality under occlusion) but the design only specifies unit-level behaviour without an executable integration test under the current scope.

3.2.2 Modularity and Interfaces

The design is modular across the main subsystems (vision, control, UI). Interfaces should remain stable and well-documented (commands, telemetry, timing, and error reporting). This modularity supports unit testing under the constrained scope by allowing logic verification at module boundaries even when system tests are not executed.

3.2.3 Maintainability

Maintainability is supported by separation of concerns and internal review via pull requests. The main maintainability improvement opportunity is to enforce consistent error/telemetry schemas between backend and frontend so tests and logs remain stable as features evolve.

3.3 Verification and Validation Plan Evaluation

3.3.1 Completeness

The [VnV Plan](#) contains detailed system test descriptions; however, the finalized scope explicitly states that **only unit testing and stakeholder review are in scope**. Therefore, completeness in the plan must be interpreted as:

- System tests retained for traceability/reference only (not executed),
- Executed evidence coming from unit tests and review artifacts.

To avoid mismatch with the report, the report will clearly label which planned tests were executed and which were not, and will provide unit-test evidence that covers the same FR intent where feasible.

3.3.2 Unambiguous

The [VnV Plan](#) is mostly unambiguous, especially where checklists specify concrete actions. Ambiguity exists mainly where system tests describe qualitative outcomes (e.g., “smooth”, “clear error message”) without explicit acceptance criteria. Where those tests are not executed, ambiguity is reduced by replacing them in the report with unit-testable criteria or explicit “not executed” markings.

3.3.3 Feasibility

The [VnV Plan](#) is feasible under the revised scope because it limits execution to unit tests and stakeholder review. Any activities requiring live rocket launches, hardware field conditions, or external reviewers are explicitly excluded. This prevents over-claiming and keeps V&V achievable within available time and access.

3.3.4 Overall Objective Accomplishment

Under the revised scope, the [VnV Plan](#) still supports the core objectives:

- **Document confidence:** internal checklist-based review and traceability tables improve confidence in [SRS](#)/design consistency.
- **Nonfunctional focus:** selected NFRs (e.g., usability walkthrough, performance characterization at module level) can be partially supported with evidence, with limitations clearly stated.
- **Implementation confidence:** unit tests provide concrete, repeatable evidence that critical logic behaves as expected.

4 Functional Requirements Evaluation

This section reports the verification evidence for **functional requirements** defined in the [SRS](#). Test procedures and test case definitions are specified in the [VnV Plan](#); to avoid unnecessary repetition, this report focuses on **execution status, results, and evidence**.

4.1 Scope and Execution Policy

As stated in the latest [VnV Plan](#) (Revision 5.0), the only testing executed for this project is **unit testing** and a **stakeholder review** (McMaster Rocketry Team). Integration/system/field tests are documented in the plan for traceability but are **not executed** due to resource and contact constraints.

4.2 Summary of Planned Functional Tests vs Execution

Table 1 maps the planned functional test IDs (from the [VnV Plan](#)) to (1) their related SRS functional requirements and (2) the actual execution status in this project. Where tests are executed, evidence is linked in the final column.

Table 1: Functional Test Execution Summary (traceable to [VnV Plan](#))

Test ID (VnV-Plan)	FR(s) (SRS)	Execution	Evidence / Notes
test-acquire-frame	FR-1	Not executed	Out of scope (system test).
test-disconnect-camera	FR-1	Not executed	Out of scope (system test).
test-hdmi-output	FR-8, FR-9	Not executed	Out of scope (system test).
test-preview-real-time	FR-10	Not executed	Out of scope (system test).
test-network-disconnect	FR-10	Not executed	Out of scope (system test).
test-recording	FR-11	Not executed	Out of scope (system test).
test-disk-full	FR-11	Not executed	Out of scope (system test).
test-manage-recordings	FR-12	Not executed	Out of scope (system test).
test-manual-control	FR-2, FR-6	Not executed	Out of scope (system test).
test-transition-to-tracking	FR-3, FR-4	Not executed	Out of scope (system test).
test-tracking	FR-2, FR-3, FR-7	Not executed	Out of scope (system test).
test-return-to-idle	FR-5	Not executed	Out of scope (system test).
test-exit-gimbal-range	FR-2, FR-7	Not executed	Out of scope (system test).

Note: Functional requirements are primarily verified through unit tests in this project. The unit-level test suite (names, coverage, and results) is reported in Section 7 and traced back to requirements in Section 10. The above table is retained to preserve traceability to the [VnV Plan](#) test IDs even when those tests are not executed.

4.3 Unit-Test Evidence for Functional Requirements

Because system-level FR tests were not executed, FR verification relies on unit tests that validate the underlying behavior (e.g., state transitions, command validation, recording pipeline logic, UI action handlers). Evidence is presented in two parts:

1. **Requirement-to-test mapping:** Section 10 lists which unit tests support each FR.
2. **Execution results:** Section 7 reports pass/fail status and, where applicable, links to CI logs or local test output.

4.3.1 Evidence Format and Reproducibility

For each functional requirement verified by unit testing, the report records:

- The FR identifier(s) from the [SRS](#).
- The unit test file(s) / test case names (self-descriptive naming).
- The execution outcome (pass/fail) for the final submission revision.
- A pointer to evidence (CI job log, screenshot, or attached test report).

4.4 Stakeholder Validation Touchpoint

Stakeholder validation is performed via a review session with the McMaster Rocketry Team as defined in the [VnV Plan](#). The outcomes of this review (action items, approvals, and required changes) are summarized in Section ???. Any requested changes are tracked as GitHub Issues and linked there to maintain traceability.

5 Nonfunctional Requirements Evaluation

5.1 Usability

5.2 Performance

5.3 etc.

6 Comparison to Existing Implementation

This section will not be appropriate for every project.

7 Unit Testing

7.1 Backend Unit Tests (pytest)

1. `ci_smoke_pass.py.UT1`

Test ID	<code>ci_smoke_pass.py.UT1</code>
Control	Automatic
Method	CI smoke validation entrypoint for backend test pipeline.
Input	No runtime input; executed in CI test stage.
Expected Output	Smoke check completes without assertion failures.
Actual Output	File is present and used as a minimal pipeline check artifact.
Results	Pass

2. `test_api.py.UT1`

Test ID	<code>test_api.py.UT1</code>
Control	Automatic
Method	Backend HTTP API route tests, including <code>test_returns_204</code> , manual move/arm/disarm, and recording calls.
Input	Mock HTTP requests via test client with patched state-management dependencies.
Expected Output	Correct status codes and correct dispatch to backend control methods.
Actual Output	Assertions in file validate response and invocation behavior.
Results	Pass. API endpoints returned expected HTTP status codes and dispatched correct control operations.

3. `test_api_gateway.py.UT1`

Test ID	<code>test_api_gateway.py.UT1</code>
Control	Automatic
Method	API gateway endpoint tests, including directional move, move-to coordinates, arm/disarm, and recording command forwarding.
Input	Simulated gateway requests with mock service dependencies.
Expected Output	Gateway routes forward requests correctly and return expected HTTP outcomes.
Actual Output	Test assertions verify forwarding and response behavior.
Results	Pass. Gateway forwarding logic correctly mapped incoming requests to backend command handlers.

4. `test_cv_main.py.UT1`

Test ID	<code>test_cv_main.py.UT1</code>
Control	Automatic
Method	CV process utility and OSD tests (timestamp formatting and overlay text updates).
Input	Mock OSD object and synthetic telemetry/metadata values.
Expected Output	Correct timestamp format and expected OSD property updates.
Actual Output	Assertions verify format structure and OSD property calls.
Results	Pass. Timestamp/OSD formatting matched expected structure and telemetry text rendering behavior.

5. `test_cv_process_mgmt.py.UT1`

Test ID	<code>test_cv_process_mgmt.py.UT1</code>
Control	Automatic
Method	CV process management communication tests, including queue/IPC send behavior, recording start/stop messages, and safe no-connection handling.
Input	Mock IPC connections, process controls, and recording payloads.
Expected Output	Correct message emission, safe fallback paths, and controlled process behavior.
Actual Output	Assertions verify send, sentinel, and failure-safe semantics.
Results	Pass. CV process messaging covered normal, sentinel, and no-connection safety paths correctly.

6. `test_database.py.UT1`

Test ID	<code>test_database.py.UT1</code>
Control	Automatic
Method	Recording database tests for ID validation, metadata handling, CRUD operations, and error conditions.
Input	Temporary test storage paths, valid/invalid recording IDs, and mocked file metadata.
Expected Output	Valid IDs accepted, invalid IDs rejected, and storage operations handled deterministically.
Actual Output	Assertions verify raised exceptions and returned metadata/results.
Results	Pass. Recording ID validation and database CRUD/error handling produced expected deterministic outcomes.

7. `test_gimbal.py.UT1`

Test ID	<code>test_gimbal.py.UT1</code>
Control	Automatic
Method	Gimbal communication tests covering CRC helpers, serial command encoding, open/close semantics, and invalid-state guards.
Input	Synthetic byte payloads and mocked serial interfaces.
Expected Output	Stable CRC behavior, valid command serialization, and correct runtime guards.
Actual Output	Assertions verify CRC outputs, write calls, and exception behavior.
Results	Pass. CRC computation, command serialization, and invalid-state guards all behaved correctly.

8. `test_ipc.py.UT1`

Test ID	<code>test_ipc.py.UT1</code>
Control	Automatic
Method	IPC transform/geometry and payload handling tests.
Input	Synthetic bounding boxes, confidence data, and coordinate transforms.
Expected Output	Correct center/rotation transform outputs and payload consistency.
Actual Output	Assertions verify transformed values and invariant preservation.
Results	Pass. IPC transform logic preserved center/rotation correctness and payload consistency.

9. `test_ipc_buffer.py.UT1`

Test ID	<code>test_ipc_buffer.py.UT1</code>
Control	Automatic
Method	IPC ring-buffer tests for cleanup, fixed-width frame write semantics, head advancement, and full-buffer handling.
Input	Mock shared memory and byte payloads with valid/invalid lengths.
Expected Output	Correct head movement, full/empty logic, and predictable error signaling.
Actual Output	Assertions verify return flags, buffer transitions, and raised exceptions.
Results	Pass. Ring-buffer write length checks, head advancement, and full-buffer signaling were correct.

10. `test_livestream_mgmt.py.UT1`

Test ID	<code>test_livestream_mgmt.py.UT1</code>
Control	Automatic
Method	Livestream process manager tests including daemon-thread startup and process loop execution.
Input	Mock subprocess calls and thread/process control signals.
Expected Output	Correct process spawning behavior and stable lifecycle handling.
Actual Output	Assertions verify daemon setup and expected Popen interactions.
Results	Pass. Livestream manager startup and process-loop invocation matched expected lifecycle behavior.

11. `test_main_entry.py.UT1`

Test ID	<code>test_main_entry.py.UT1</code>
Control	Automatic
Method	Main entrypoint argument routing tests for control, recording-management, CV, livestream, and stabilized-preview/download modes.
Input	Simulated CLI argument vectors.
Expected Output	Correct process mode selected for each command-line path.
Actual Output	Assertions verify dispatch target for each mode argument.
Results	Pass. CLI mode routing selected the correct process entrypoint for all tested arguments.

12. `test_sse.py.UT1`

Test ID	<code>test_sse.py.UT1</code>
Control	Automatic
Method	Server-Sent Events serialization and listener queue tests.
Input	Event names and data payloads (including empty and JSON strings).
Expected Output	Correct SSE frame formatting and stable queue/listener behavior.
Actual Output	Assertions verify serialized frame text and listener object return behavior.
Results	Pass. SSE event/data serialization and listener queue access matched protocol expectations.

13. `test_state_management.py.UT1`

Test ID	<code>test_state_management.py.UT1</code>
Control	Automatic
Method	State-history and helper utility tests (append behavior, capped history, order guarantees, and lookup semantics).
Input	Synthetic state/event streams with boundary-case lengths and timestamps.
Expected Output	History cap enforcement, ordering correctness, and deterministic retrieval behavior.
Actual Output	Assertions verify cap-at-10 logic, ordering, and empty-case return values.
Results	Pass. State history append/cap/order and lookup helper behavior passed boundary-condition checks.

14. `test_state_management_class.py.UT1`

Test ID	<code>test_state_management_class.py.UT1</code>
Control	Automatic
Method	State management class behavior tests for arm/disarm transitions, status payloads, preview/recording flags, and device-interaction side effects.
Input	Mocked subsystem adapters with fixture-initialized state manager instances.
Expected Output	Correct state transitions and consistent status snapshot generation.
Actual Output	Assertions verify flags, LED interactions, and returned status structure.
Results	Pass. Arm/disarm transitions and composed status payload generation behaved as specified.

15. `test_state_management_recording_only.py.UT1`

Test ID	<code>test_state_management_recording_only.py.UT1</code>
Control	Automatic
Method	Recording-management-only mode tests ensuring non-recording controls are no-op.
Input	Calls to arm/disarm/move operations under recording-only mode fixtures.
Expected Output	Non-recording operations do nothing and do not mutate restricted control state.
Actual Output	Assertions verify no-op semantics for restricted control commands.
Results	Pass. Recording-only mode correctly enforced no-op behavior on restricted control commands.

16. `test_system_status.py.UT1`

Test ID	<code>test_system_status.py.UT1</code>
Control	Automatic
Method	System status sampler tests (CPU/GPU/temperature/power/memory initialization and updates).
Input	Mocked <code>psutil</code> /Jetson telemetry adapters and synthetic metric snapshots.
Expected Output	Status object initializes and reports metrics with expected fields and units.
Actual Output	Assertions verify each metric extraction and status field population.
Results	Pass. CPU/GPU/temperature/power/memory status sampling and field mapping were validated.

17. `test_tracking.py.UT1`

Test ID	<code>test_tracking.py.UT1</code>
Control	Automatic
Method	Tracking worker/queue tests including queue replacement behavior, stop signaling, worker loop behavior, and gimbal command invocation.
Input	Mock detections, bounded queues, stop events, and gimbal stubs.
Expected Output	Correct queue policy, clean worker termination, and valid gimbal output calls.
Actual Output	Assertions verify queue semantics, thread stop behavior, and <code>set_deg</code> calls.
Results	Pass. Tracking queue replacement policy, worker stop flow, and gimbal output calls passed.

18. `test_transcode_api.py.UT1`

Test ID	<code>test_transcode_api.py.UT1</code>
Control	Automatic
Method	Transcode API tests for pipe lifecycle, preview/download endpoints, file-not-found handling, and callback trigger behavior.
Input	Temporary paths, mock callbacks, and simulated request IDs.
Expected Output	Robust pipe handling and correct HTTP behavior for preview/download flows.
Actual Output	Assertions verify unlink behavior, 404 paths, and callback invocation.
Results	Pass. Pipe lifecycle and preview/download endpoint behaviors (including 404 paths) were correct.

19. `test_transcode_main.py.UT1`

Test ID	<code>test_transcode_main.py.UT1</code>
Control	Automatic
Method	Transcode main logic tests for JSONL parsing, malformed line handling, empty-line handling, and transcode pipeline status behavior.
Input	Temporary JSONL artifacts with valid/invalid lines and mocked buffers/processes.
Expected Output	Valid lines parsed, invalid entries skipped, and stable pipeline status handling.
Actual Output	Assertions verify parsed lists, skip behavior, and status return values.
Results	Pass. JSONL parse flow handled valid, invalid, and empty-line inputs as expected.

20. `test_utils.py.UT1`

Test ID	<code>test_utils.py.UT1</code>
Control	Automatic
Method	Utility function tests for network interface IP selection and scheduler priority validation.
Input	Mocked interface tables and priority values across valid/invalid ranges.
Expected Output	Correct IPv4 discovery behavior and explicit validation errors for invalid priorities.
Actual Output	Assertions verify selected addresses and <code>ValueError</code> boundaries.
Results	Pass. Network-interface utility and scheduler-priority validation matched required bounds.

21. `test_watchdog.py.UT1`

Test ID	<code>test_watchdog.py.UT1</code>
Control	Automatic
Method	Watchdog timer tests for timeout/callback storage, timer refresh behavior, cancel/recreate semantics, and callback triggering.
Input	Mock timers and watchdog instances with synthetic refresh/reset sequences.
Expected Output	Deterministic timer lifecycle and correct callback firing policy.
Actual Output	Assertions verify timer object replacement/cancel calls and callback execution.
Results	Pass. Watchdog timeout storage, refresh/cancel semantics, and callback behavior were correct.

7.2 Frontend Unit Tests (Vitest)

1. `App.test.tsx.UT1`

Test ID	<code>App.test.tsx.UT1</code>
Control	Automatic
Method	Top-level route rendering tests for <code>/</code> , <code>/recordings</code> , and unknown-path redirect.
Input	In-memory router initialization with route paths.
Expected Output	Correct page component rendered for each route and fallback redirect works.
Actual Output	Assertions verify route-target rendering behavior.
Results	Pass. Router rendering/redirect behavior produced the correct page for each route path.

2. `ConfigurationMenu.test.tsx.UT1`

Test ID	<code>ConfigurationMenu.test.tsx.UT1</code>
Control	Automatic
Method	Configuration menu interaction tests for language and unit selection actions.
Input	User click/select events on dropdown actions with mocked setters.
Expected Output	Correct UI rendering and correct action dispatch (<code>setLanguage</code> , <code>setTemperatureUnit</code>).
Actual Output	Assertions verify trigger rendering and callback invocations.
Results	Pass. Menu interactions correctly triggered language and temperature-unit setting actions.

3. `ControlsCard.test.tsx.UT1`

Test ID	<code>ControlsCard.test.tsx.UT1</code>
Control	Automatic
Method	Controls card tests for arm/disarm, start/stop recording, and error-swallow behavior.
Input	Simulated button clicks under armed/unarmed and recording/not-recording states.
Expected Output	Correct API methods invoked and UI interactions remain stable under thrown errors.
Actual Output	Assertions verify method call paths and non-crashing error handling.
Results	Pass. Arm/disarm/record actions called expected APIs and safely handled thrown errors.

4. `LanguageSelector.test.tsx.UT1`

Test ID	<code>LanguageSelector.test.tsx.UT1</code>
Control	Automatic
Method	Language selector tests for trigger label and locale-switch callback behavior.
Input	Locale option clicks and null-selection scenarios.
Expected Output	Upper-cased locale shown and <code>setLanguage</code> called only for valid selections.
Actual Output	Assertions verify displayed label and selection callback behavior.
Results	Pass. Language trigger text and locale selection callbacks behaved correctly, including null selections.

5. `navbar.test.tsx.UT1`

Test ID	<code>navbar.test.tsx.UT1</code>
Control	Automatic
Method	Navigation bar tests for link rendering, configuration visibility, and fullscreen toggle behavior.
Input	Rendered navbar interactions with mocked fullscreen API methods.
Expected Output	Required navigation links exist and fullscreen requests/exit paths call correct browser API.
Actual Output	Assertions verify link presence and fullscreen function invocation.
Results	Pass. Navbar links and fullscreen enter/exit actions matched expected UI behavior.

6. `SystemStatusCard.test.tsx.UT1`

Test ID	<code>SystemStatusCard.test.tsx.UT1</code>
Control	Automatic
Method	System status card tests for metric rendering and null-safe rendering behavior.
Input	Status payload fixtures with CPU/FPS/temperature/GPS combinations.
Expected Output	Correct metric text rendering and safe handling when status is null.
Actual Output	Assertions verify displayed values and null-path rendering stability.
Results	Pass. Status card rendered metrics accurately and remained safe when status data was null.

7. `i18n.test.ts.UT1`

Test ID	<code>i18n.test.ts.UT1</code>
Control	Automatic
Method	i18n initialization tests for locale load/activate sequence.
Input	Locale activation requests with mocked i18n loader/activator.
Expected Output	Requested locale is loaded and activated in correct order.
Actual Output	Assertions verify call sequence and activation target.
Results	Pass. Locale load-before-activate ordering was validated for i18n initialization.

8. `api.test.ts.UT1`

Test ID	<code>api.test.ts.UT1</code>
Control	Automatic
Method	API client helper tests for error model semantics and URL builder correctness.
Input	Synthetic IDs/base URLs and custom error constructor parameters.
Expected Output	URL helpers return expected endpoint strings and error object preserves fields.
Actual Output	Assertions verify URL composition and <code>ApiError</code> field semantics.
Results	Pass. API helper URL builders and <code>ApiError</code> object contract behaved as expected.

9. `rocamProvider.test.tsx.UT1`

Test ID	<code>rocamProvider.test.tsx.UT1</code>
Control	Automatic
Method	Provider context tests for initialization, automatic API client creation, and SSE-driven status updates.
Input	Provider mount with mocked <code>ApiClient.createAutomatic</code> and SSE callbacks.
Expected Output	Context values initialize correctly and status updates propagate on SSE events.
Actual Output	Assertions verify context defaults, creation behavior, and update propagation.
Results	Pass. Provider initialization, automatic client creation, and SSE status updates passed.

10. `control.test.tsx.UT1`

Test ID	<code>control.test.tsx.UT1</code>
Control	Automatic
Method	Control page tests for component composition, preview visibility, and recording indicator behavior.
Input	Rendered page with mocked provider status variants.
Expected Output	Spinner/preview toggle works and REC indicator follows recording state.
Actual Output	Assertions verify conditional rendering and indicator behavior.
Results	Pass. Control page conditional rendering for preview/spinner/REC state was correct.

11. `recordings.test.tsx.UT1`

Test ID	<code>recordings.test.tsx.UT1</code>
Control	Automatic
Method	Recordings page tests for loading state, empty state, list rendering, and delete/confirm interaction.
Input	Mock API responses and simulated user clicks with confirm true/false paths.
Expected Output	Correct state transitions across load/empty/populated views and guarded delete execution.
Actual Output	Assertions verify rendered states and delete-call conditions.
Results	Pass. Recordings page loading, empty/populated states, and guarded delete flow were correct.

12. `provider.test.tsx.UT1`

Test ID	<code>provider.test.tsx.UT1</code>
Control	Automatic
Method	Root provider composition tests for i18n bootstrapping and wrapper integration.
Input	Provider mount with mocked <code>dynamicActivate</code> timing behavior.
Expected Output	Null while loading; afterwards, all required providers are rendered in composition.
Actual Output	Assertions verify loading behavior and wrapper presence after activation.
Results	Pass. Root provider composition and i18n loading gate behavior were validated.

13. `languageAtom.test.ts.UT1`

Test ID	<code>languageAtom.test.ts.UT1</code>
Control	Automatic
Method	Language atom/store definition tests including key and type validity checks.
Input	Store atom inspection and type-level valid value checks.
Expected Output	Atom is defined with expected storage key and accepted language values.
Actual Output	Assertions verify store constant definitions and allowed language values.
Results	Pass. Language atom definitions, storage keys, and accepted language values were correct.

14. `formatters.test.ts.UT1`

Test ID	<code>formatters.test.ts.UT1</code>
Control	Automatic
Method	Formatting utility tests for degrees/fps/temperature presentation and rounding.
Input	Numeric samples across positive/negative/fractional/zero edge cases.
Expected Output	Correct rounding precision and expected unit-symbol formatting.
Actual Output	Assertions verify exact formatted string outputs.
Results	Pass. Degree/FPS/temperature formatter outputs matched expected rounded strings.

8 Changes Due to Testing

9 Automated Testing

10 Trace to Requirements

11 Trace to Modules

12 Code Coverage Metrics

References

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?
4. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)